Advanced Machine learning Mastering Course

Introduced by

George Samuel
PhD researcher

Introduction into NLP

Innovisionray.com

Machine Learning Diploma

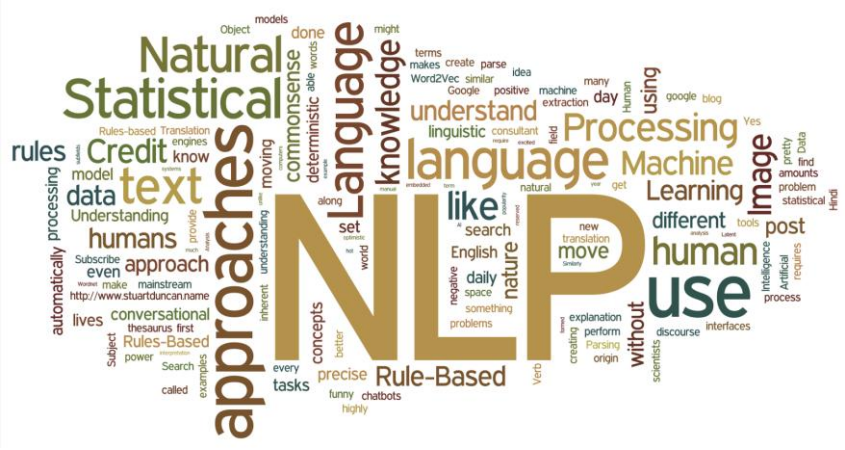
Natural language processing

Agenda:

1	What is NLP ?
2	Tokenization
3	NLTK
4	Stop words
5	Stemming
6	lemmatization
7	Regular expression (Regex)

8	Part of speech
9	Named Entity recognition (NER)
10	Spell correction
11	Bag of words
12	N-GRAM
13	TF-IDF
14	One hot encoding
15	Word Embedding

1- What is natural language processing (NLP)?



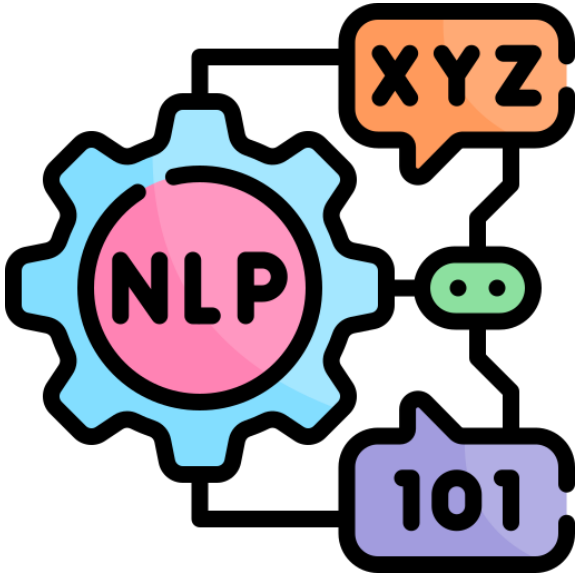
- Natural language processing (NLP) is a type of artificial intelligence that helps machines understand human language. It's responsible for many tools that make our lives easier.

- We know that models can't directly work with text, so I need to convert this text into numbers. There are techniques for doing this, like Bag of Words, TF-IDF, and Word Embedding.

- The word “**Corpus**" refers to paragraph, while “**Tokens**" are individual words.

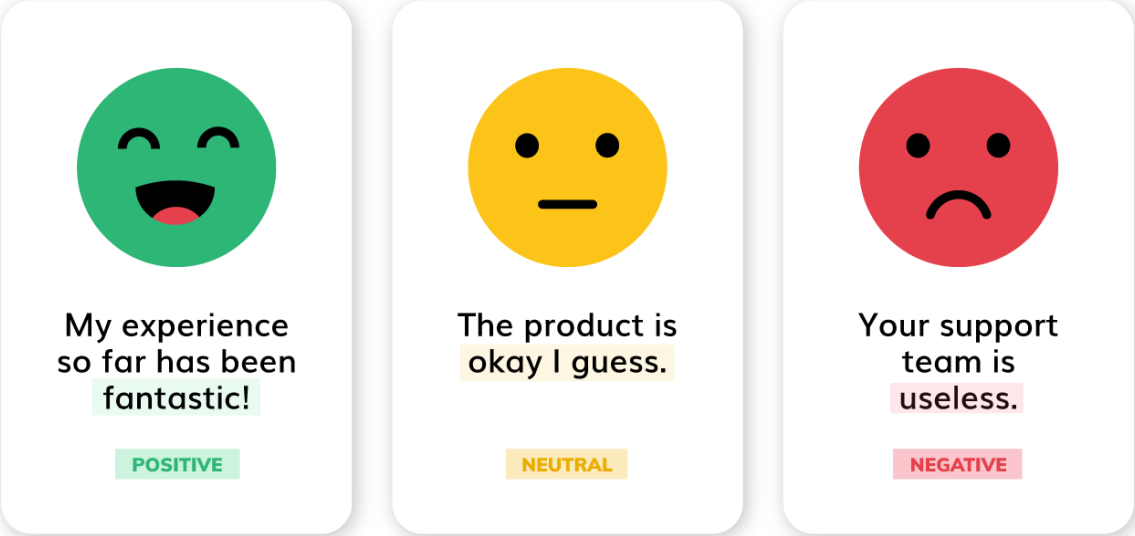
What are the applications of NLP?

1. Sentiment Analysis
2. Chatbots
3. Machine Translation
4. Text Summarization
5. Auto-Correct



Sentiment Analysis :

Sentiment analysis is a type of natural language processing (NLP) that falls under text classification. In simple terms, it's about figuring out the feeling behind a text, like whether it's positive, negative, happy, sad, or neutral. The main aim is to understand the mood or emotion of the text.



Chatbots :

An NLP chatbot is a computer programmer that talks with people using words like humans do. It learns how to understand and reply to human language by using special techniques.

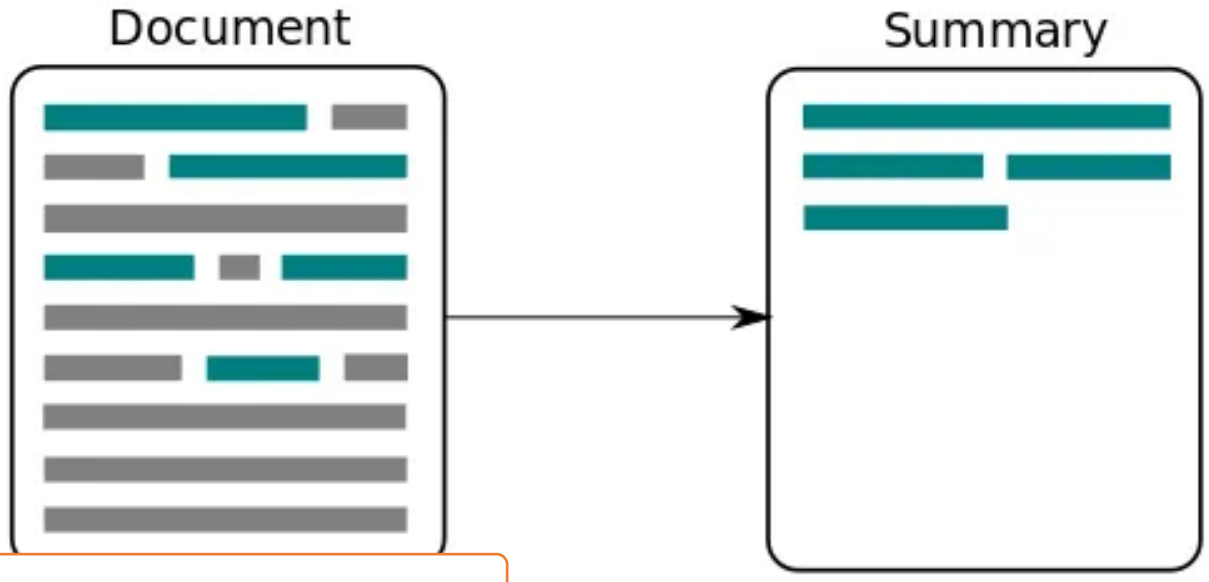
example: **chatgpt**

Machine Translation :

Machine Translation is the process of converting text from one language to another, considering context and grammar. **Google Translate** is a popular tool for this. If you travel, you've probably used the Google Translate app.

Text Summarization :

Text summarization means making a short and easy-to-read summary of a longer text. The goal is to find the most important information and present it in a clear way. This is useful because it helps people quickly understand a large amount of information.



Auto-Correct :

Autocorrect, also known as automatic correction, uses Natural Language Processing (NLP) to fix spelling and typing mistakes in text. It's designed to make writing easier by automatically correcting errors as you type.

2- Tokenization

Tokenization:

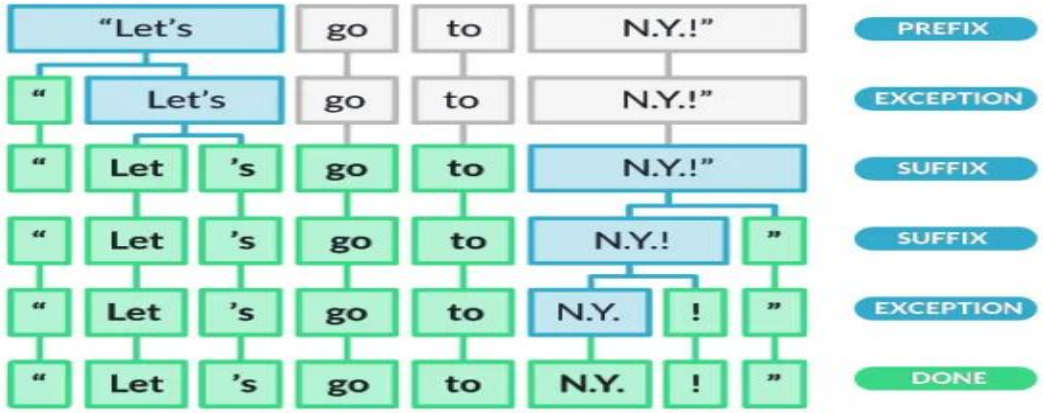
- Tokenization is the process of dividing a corpus into words, which is called **word tokenization**, or into sentences, which is called **sentence tokenization**

- This function **breaks down text** using punctuation like distance or commas. It then adds **prefixes** (separating anything before a word, e.g., double quotation), exceptions (separating words stuck together, e.g., "let's"), and suffixes (separating anything after a word).

- The tokenizer is clever. It knows to treat special cases like "N.Y" as one word because it recognizes it as "New York." But it can separate words like "let's" because it's actually two words. However, functions like split will treat them as one word.

- **Words came out in the end, we call them tokens.**

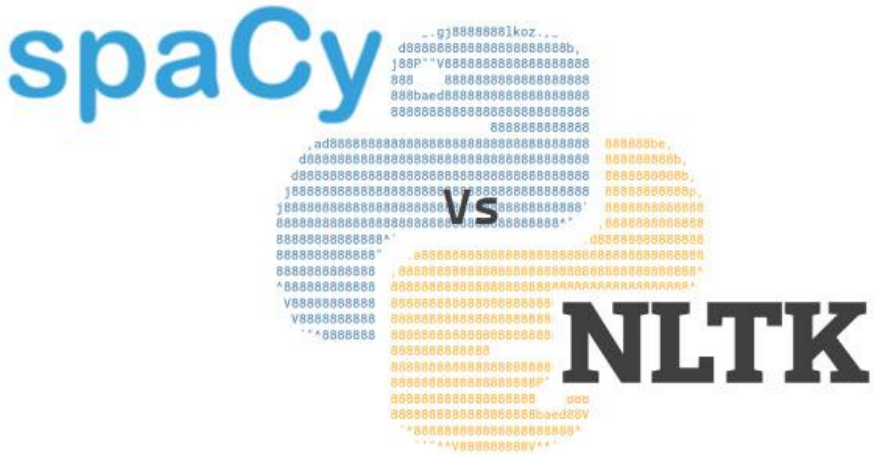
https://platform.openai.com/tokenizer



3- NLTK

NLTK:

- It's one of the libraries used in NLP, like Sklearn in ML. There's another important library called Scipy.



4- Stop words

What are Stop Words ?

- These are words that appear frequently in speech but are less important than others, like "and," "or," "I," and "she.“

- So, we erase it for two reasons: first, to save time for the model, and second, so it can focus on the rest of the words.

Stop words are words that are commonly used in a language but are considered to have little or no importance in text analysis and natural language processing (NLP) tasks. These words are typically filtered out from text data because they do not carry much meaning and can clutter up the analysis.

In English, some examples of stop words include "the", "and", "a", "an", "of", "in", "is", "it", "to", "on", "that", "this", "with", "as", "at", and "for".

Removing stop words from text data is a common pre-processing step in many NLP tasks, such as text classification, sentiment analysis, and topic modeling. By removing these words, the resulting text data can be more focused and meaningful, making it easier to extract useful insights and information.

5- Stemming

Stemming :

- It means that removing the prefix and suffix of a word brings it back to its basic form. For example, both "runs" and "running" are derived from the same word, which is "run."

- "What's the benefit? We're building data based on a large corpus, so I need to extract only unique words. For example, if the word is 'running,' I record it in my data more than once for 'running, runs, run,' but we link them back to the original word, which is 'run.'"

Stemming

Stemming is a natural language processing (NLP) technique used to reduce words to their root or base form, which is called the "stem." Stemming is commonly used as a pre-processing step in text mining and information retrieval tasks, such as document classification, clustering, and search engines.

The stem is the part of the word that remains after removing any prefixes or suffixes. For example, the word "running" can be reduced to its stem "run" by removing the suffix "-ing". Similarly, the word "cats" can be reduced to its stem "cat" by removing the suffix "-s".

Stemming :

- have 3 types of stemming, each type differs from the next in that it is more **aggressive**, meaning it applies rules more to reduce the number of unique words it returns.

1. Porter Stemming Algorithm
2. Snowball Stemming
3. Lancaster Stemming

Stemming

Porter Stemming

It is less aggressive in stemming and keeps more of the original word than some of the other algorithms.

Snowball Stemming

This algorithm is more aggressive and efficient than the Porter algorithm. It applies a larger set of rules to reduce words to their base form.

Lancaster Stemming

This algorithm is the most aggressive among the three. It uses many rules to reduce a word to its base form.

5- lemmatization

Lemmatization:

- Stemming has a problem in that it returns the word without understanding the context or the word's meaning, so it produces many meaningless things.

- Lemmatization has become more efficient because it considers the context and the original meaning, which is why it is being used more frequently nowadays.

- Does that mean we won't use stemming at all? In some cases where context doesn't matter much, you can use it.

Word	Stemming	Lemmatization
information	inform	information
informative	inform	informative
computers	comput	computer
feet	feet	foot

6- Regular expression

Regular expression :

- We use it to extract a specific pattern from the text.
- This can be implemented in vision using something called optical character recognition (OCR).
- Of course, it can't be saved, so that's why we're going to this website **https://regex101.com/**

Regular expression :

How can I import the library ?

```
import re
```

Add Item To List

findall()

Returns a list containing all matches

sub()

Returns a Match object if there is a match anywhere in the string

search()

Returns a list where the string has been split at each match

split()

Replaces one or many matches with a string

The findall() Function:

The findall() function returns a list containing all matches.

```
import re

txt = "The Amit in Egypt is one of the best places.(Amit)"
x = re.findall("Amit", txt)
print(x)
```

```
['Amit', 'Amit']
```

If no matches are found, **an empty list is returned**

```
import re

txt = "The AI for everyone"
x = re.findall("amit", txt)
print(x)
```

```
[]
```

The sub() Function :

The sub() function replaces the matches with the text of your choice

```
import re

txt = "The Amit in Egypt is one of the best places.(Amit)"
x = re.sub("o", "9", txt)
print(x)
```

```
The Amit in Egypt is 9ne 9f the best places.(Amit)
```

You can control the number of replacements by specifying the count parameter

```
import re

txt = "The Amit in Egypt is one of the best places.(Amit)"
x = re.sub("o", "9", txt, 1)
print(x)
```

```
The Amit in Egypt is 9ne of the best places.(Amit)
```

The search() Function:

The search() function searches the string for a match, and returns a Match object if there is a match.If there is more than one match, only the first occurrence of the match will be returned:

```
import re

txt = "The rain in Spain"
x = re.search("\s", txt)

print("The first white-space character is located in position:", x.start())
```

```
The first white-space character is located in position: 3
```

If no matches are found, the value None is returned:

```
import re

txt = "The rain in Spain"
x = re.search("Portugal", txt)
print(x)
```

```
None
```

The split() Function:

The split() function returns a list where the string has been split at each match

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

```
['The', 'rain', 'in', 'Spain']
```

You can control the number of occurrences by specifying the maxsplit parameter

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt, 1)
print(x)
```

```
['The', 'rain in Spain']
```

Match ranges in regex :

Regex uses ranges to match sequential groups of characters, like uppercase English letters, without having to write out each letter individually. This makes it easier and less tedious to specify such patterns in your expressions.

RANGE	MATCHES
[A-Z]	uppercase letters
[a-z]	lowercase letters
[0-9]	Any digit

Examples

```python
import re
input_string = 'Hello World 123'

uppercase_letters = re.findall(r'[A-Z]', input_string)
print("Uppercase letters:", uppercase_letters)
```

```
Uppercase letters: ['H', 'W']
```

```python
import re
input_string = 'Hello World 123'

# Digits
digits = re.findall(r'[0-9]', input_string)
print("Digits:", digits)
```

```
Digits: ['1', '2', '3']
```

```python
import re
input_string = 'Hello World 123'

# Lowercase letters
lowercase_letters = re.findall(r'[a-z]', input_string)
print("Lowercase letters:", lowercase_letters)
```

```
Lowercase letters: ['e', 'l', 'l', 'o', 'o', 'r', 'l', 'd']
```

RANGE	MATCHES
.	anything except newline
\w	word character
\W	non-word character
\d	digits
\D	non-digits
\s	space, tab, newlines
\S	non whitespace characters

Examples

```python
import re


text = "Hello World_123"

pattern = re.compile(r'\w')
matches = pattern.findall(text)
print("Matches any word character :",matches)
```

```
Matches any word character : ['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', '_', '1', '2', '3']
```

```python
import re

# Sample text
text = "Hello 123 World 456!"

# Match non-digits
pattern = re.compile(r'\D')
matches = pattern.findall(text)
print(matches)
```

```
['H', 'e', 'l', 'l', 'o', ' ', ' ', 'W', 'o', 'r', 'l', 'd', ' ', '!']
```

```python
import re

# Sample text
text = "Hello World_123!"

# Match non-word characters
pattern = re.compile(r'\W')
matches = pattern.findall(text)
print("Match non-word characters",matches)
```

```
Match non-word characters [' ', '!']
```

```python
import re

# Sample text
text = "Hello 123 World 456"

# Match digits
pattern = re.compile(r'\d')
matches = pattern.findall(text)
print(matches)
```

```
['1', '2', '3', '4', '5', '6']
```

Examples

```python
import re

# Sample text
text = "Hello World Python"

# Match non-whitespace characters
pattern = re.compile(r'\S')
matches = pattern.findall(text)
print(matches)
```

```
['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', 'P', 'y', 't', 'h', 'o', 'n']
```

```python
import re

# Sample text
text = "Hello\tWorld\nPython"

# Match whitespace characters
pattern = re.compile(r'\s')
matches = pattern.findall(text)
print(matches)
```

```
['\t', '\n']
```

```python
import re
text = "Hello, world\n!"
pattern = r"."
matches = re.findall(pattern, text)
print(matches)
```

```
['H', 'e', 'l', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'l', 'd', '!']
```

RANGE	MATCHES
*	0 or more
?	0 or 1
+	1 or more
{n}	exactly n times
{n, }	n or more times
{n, m}	n to m times inclusive

Examples

```python
import re

pattern = r"ab*"
test_string = "ababab"
result = re.findall(pattern, test_string)
print(result)  # Output: ['ab', 'ab', 'ab']
```

```
['ab', 'ab', 'ab']
```

```python
import re

pattern = r"ab+"
test_string = "abb"
result = re.findall(pattern, test_string)
print(result)  # Output: ['abb']
# +: Matches one or more occurrences of the preceding element.
```

```
['abb']
```

```python
import re

pattern = r"ab{2,}"
test_string = "abbbbb"
result = re.findall(pattern, test_string)
print(result)  # Output: ['abbbbb']
# {n,}: Matches n or more occurrences of the preceding element.
```

```
['abbbbb']
```

```python
import re

pattern = r"ab?"
test_string = "a"
result = re.findall(pattern, test_string)
print(result)  # Output: ['a']
#?: Matches zero or one occurrence of the preceding element.
```

```
['a']
```

```python
import re

pattern = r"ab{2}"
test_string = "abb"
result = re.findall(pattern, test_string)
print(result)  # Output: ['abb']
# {n}: Matches exactly n occurrences of the preceding element.
```

```
['abb']
```

```python
import re

pattern = r"ab{2,3}"
test_string = "abbb"
result = re.findall(pattern, test_string)
print(result)  # Output: ['abb']
#{n, m}: Matches between n and m occurrences of the preceding element, inclusive.
```

```
['abbb']
```

Try To Solve

Write a regular expression to match a phone number in the format "(XXX) XXX-XXXX"

```
import re

phone_number_pattern = r"\(\d{3}\) \d{3}-\d{4}"
test_string = "Call me at (123) 456-7890"
matches = re.findall(phone_number_pattern, test_string)
print(matches)
```

Try To Solve

How can you extract all email addresses from a given text using regular expressions in Python?

"Contact us at email@example.com or info@example.com"

```python
import re

text = 'Contact us at email@example.com or info@example.com'
pattern = r'\b\w+@\w+\.com'
emails = re.findall(pattern, text)
print(emails)
```

Python Regular Expressions

SPECIAL CHARACTERS

`^` | Matches the expression to its right at the start of a string. It matches every such instance before each `\n` in the string.

`$` | Matches the expression to its left at the end of a string. It matches every such instance before each `\n` in the string.

`.` | Matches any character except line terminators like `\n`.

`\` | Escapes special characters or denotes character classes.

`A|B` | Matches expression `A` or `B`. If A is matched first, B is left untried.

`+` | Greedily matches the expression to its left 1 or more times.

`*` | Greedily matches the expression to its left 0 or more times.

`?` | Greedily matches the expression to its left 0 or 1 times. But if `?` is added to qualifiers (`+`, `*`, and `?` itself) it will perform matches in a non-greedy manner.

`{m}` | Matches the expression to its left `m` times, and not less.

`{m,n}` | Matches the expression to its left `m` to `n` times, and not less.

`{m,n}?` | Matches the expression to its left `m` times, and ignores `n`. See `?` above.

CHARACTER CLASSES (A.K.A. SPECIAL SEQUENCES)

`\w` | Matches alphanumeric characters, which means `a-z`, `A-Z`, and `0-9`. It also matches the underscore, `_`.

`\d` | Matches digits, which means `0-9`.

`\D` | Matches any non-digits.

`\s` | Matches whitespace characters, which include the `\t`, `\n`, `\r`, and space characters.

`\S` | Matches non-whitespace characters.

`\b` | Matches the boundary (or empty string) at the start and end of a word, that is, between `\w` and `\W`.

`\B` | Matches where `\b` does not, that is, the boundary of `\w` characters.

`\A` | Matches the expression to its right at the absolute start of a string whether in single or multi-line mode.

`\Z` | Matches the expression to its left at the absolute end of a string whether in single or multi-line mode.

SETS

`[ ]` | Contains a set of characters to match.

`[amk]` | Matches either `a`, `m`, or `k`. It does not match `amk`.

`[a-z]` | Matches any alphabet from `a` to `z`.

`[a\-z]` | Matches `a`, `-`, or `z`. It matches `-` because `\` escapes it.

`[a-]` | Matches `a` or `-`, because `-` is not being used to indicate a series of characters.

`[-a]` | As above, matches `a` or `-`.

`[a-z0-9]` | Matches characters from `a` to `z` and also from `0` to `9`.

`[(+*)]` | Special characters become literal inside a set, so this matches `(`, `+`, `*`, and `)`.

`[^ab5]` | Adding `^` excludes any character in the set. Here, it matches characters that are not `a`, `b`, or `5`.

GROUPS

`( )` | Matches the expression inside the parentheses and groups it.

`(?)` | Inside parentheses like this, `?` acts as an extension notation. Its meaning depends on the character immediately to its right.

`(?PAB)` | Matches the expression `AB`, and it can be accessed with the group name.

`(?aiLmsux)` | Here, `a`, `i`, `L`, `m`, `s`, `u`, and `x` are flags:

- `a` — Matches ASCII only
- `i` — Ignore case
- `L` — Locale dependent
- `m` — Multi-line
- `s` — Matches all
- `u` — Matches unicode
- `x` — Verbose

`(?:A)` | Matches the expression as represented by `A`, but unlike `(?PAB)`, it cannot be retrieved afterwards.

`(?#...)` | A comment. Contents are for us to read, not for matching.

`A(?=B)` | Lookahead assertion. This matches the expression `A` only if it is followed by `B`.

`A(?!B)` | Negative lookahead assertion. This matches the expression `A` only if it is not followed by `B`.

`(?<=B)A` | Positive lookbehind assertion. This matches the expression `A` only if `B` is immediately to its left. This can only matched fixed length expressions.

`(?<!B)A` | Negative lookbehind assertion. This matches the expression `A` only if `B` is not immediately to its left. This can only matched fixed length expressions.

`(?P=name)` | Matches the expression matched by an earlier group named "name".

`(...)\1` | The number `1` corresponds to the first group to be matched. If we want to match more instances of the same expression, simply use its number instead of writing out the whole expression again. We can use from `1` up to `99` such groups and their corresponding numbers.

POPULAR PYTHON RE MODULE FUNCTIONS

`re.findall(A, B)` | Matches all instances of an expression `A` in a string `B` and returns them in a list.

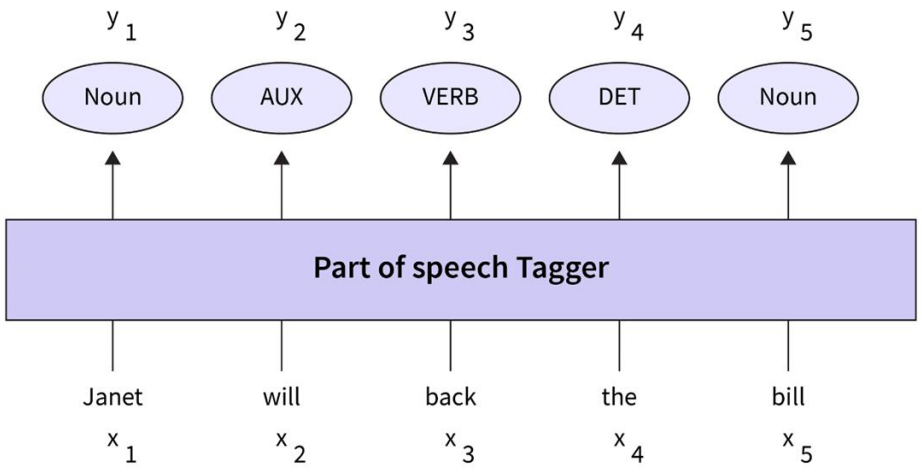
`re.search(A, B)` | Matches the first instance of an expression `A` in a string `B`, and returns it as a re match object.

`re.split(A, B)` | Split a string `B` into a list using the delimiter `A`.

`re.sub(A, B, C)` | Replace `A` with `B` in the string `C`.

George Samuel –PHD researcher- Cairo University –FSSR-Computer science

1- What is part of speech?



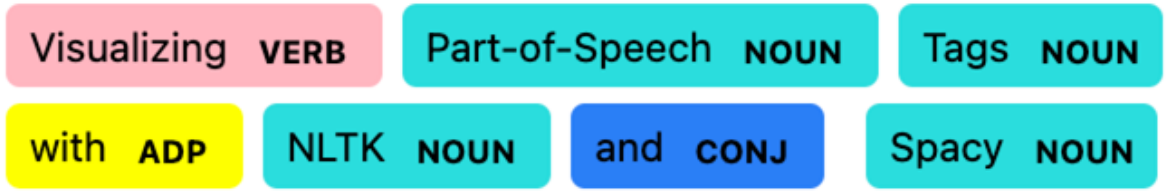
- He analyzes each word to understand the implied meaning of the speech. He can respond to you in the tense you are speaking .

- For example: noun, verb, adjective, and so on.

- You can use it for sentiment analysis, especially with exclamation marks and describing words (adjectives). These help understand if the text is positive or negative. It's also helpful in filtering. For example, you can find the type of each thing and remove things like distances and empty values. Or, if you want to gather a lot of text, you can first find pos to exclude words like pronouns.

- It provides you with the grammatical analysis of words in the form of letters. If you don't know their meanings, enter this website. https://v2.spacy.io/api/annotation

Example:



dslim/bert-base-NER · Hugging Face

2- Named Entity recognition (NER)

What is Named Entity recognition (NER) ?

Named entity recognition (NER) is a way to find and sort important information in text. It helps identify things like names, places, and dates.

When I say my name is Shnawi and I live in Mansoura and work at Photon company, he explains to me this information and tells me that Shnawi is a person, Mansoura is a place, and Photon is a company.

Example:

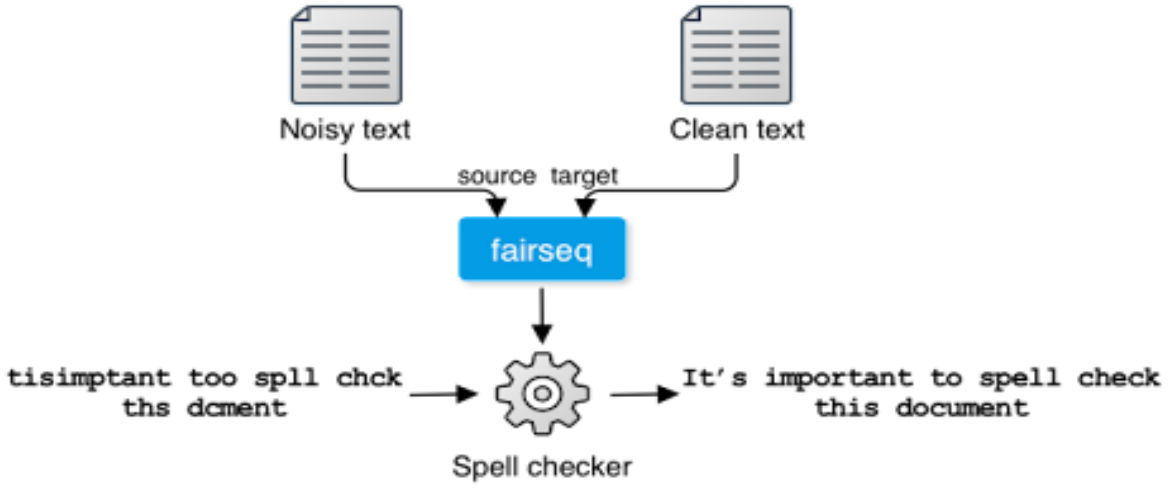
Elon Reeve Musk PERSON born June 28, 1971 DATE is a businessman and investor. He is the founder, chairman, CEO, and CTO ORG of SpaceX; angel investor PERSON , CEO, product architect, and former chairman of Tesla, Inc. ORG ; owner, chairman, and CTO of X Corp. ORG ; founder of the Boring Company ORG and xAI; co-founder of Neuralink ORG and OpenAI GPE ; and president of the Musk Foundation ORG . He is the second ORDINAL wealthiest person in the world, with an estimated net worth of US$232 billion MONEY as of December 2023 DATE , according to the Bloomberg Billionaires Index ORG , and $182.6 billion MONEY according to Forbes ORG , primarily from his ownership stakes in Tesla ORG and SpaceX

3- Spell correction

What is the spell correction ?

- Sometimes, words are misspelled, so it's important to correct them using spell correction as part of the first preprocessing step before converting words to numbers. This can be done using a spell checker or the TextBlob library.

- For any type of text processing or analysis, checking the spelling of the word is one of the basic requirements.



Spell correction

Spell checker

```python
#!pip install pyspellchecker
from spellchecker import SpellChecker
spell = SpellChecker()
misspelled = spell.unknown(["cmputr", "watr", "study", "wrte"])
for word in misspelled:
    print(spell.correction(word))
```

```
write
computer
water
```

TextBlob library

```python
### !pip install textblob
from textblob import TextBlob

a = "cmputr"            # incorrect spelling
print("original text: "+str(a))

b = TextBlob(a)

# prints the corrected spelling
print("corrected text: "+str(b.correct()))
```

```
original text: cmputr
corrected text: computer
```

Quick Recap :

This text mentions that there was some preprocessing done on the text before converting it into numbers

Tokenization: Breaking text into smaller parts, like words or sentences.

Stemming and Lemmatization: Simplifying words to their base form.

Regular Expression: A pattern used to search for and manipulate text.

POS (Part-of-Speech): Identifying the grammatical category of a word (e.g., noun, verb).

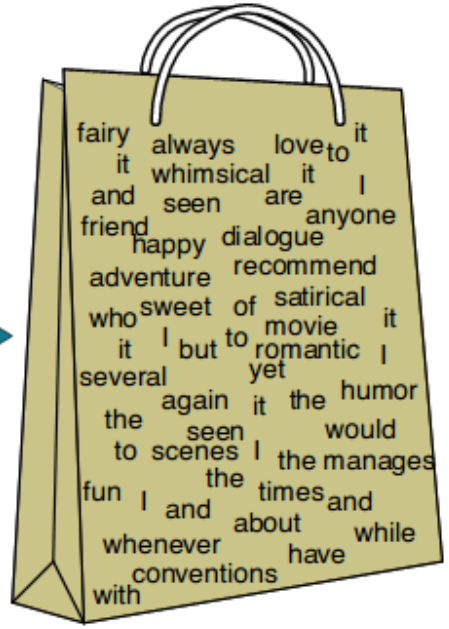
NER (Named Entity Recognition): Identifying and categorizing named entities in text (e.g., person names, locations).

Spell Correction: Fixing spelling mistakes in text.

Object
Vector
Task
Happy Birthday
TEXT
NLP TRANSFORMER
[0.3, -0.4, 1.2, 0.3, ...]
Translation, understanding, Sentiment, Question Answering, Semantic Search, ...

5- Bag of words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

What are Bag of words ?

- It is a technique used to convert text into numeric numbers .
- It is used with important tasks in NLP like text classification and sentiment analysis.
- "Very similar to one hot encoding."
- All it does is take from my corpus, which contains all the words I have, the unique words, and then converts them to vectors. So, whenever you encounter this word anywhere, you remove it and put its vector in its place.

- The order of the words here is not important and doesn't affect the formation of the vector. What matters is the frequency of each word.

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

Notes:

- It's better to convert all your letters to small before applying the BoW because if it encounters the word "he" and "He", it will put each one of them in a separate column and calculate their frequency separately.

- Also, remove the stop words because they are very repetitive, so they will have high frequency numbers, and then the model will mistakenly think that they are more important, which is wrong.

Sentence 1 → He is a good boy
Sentence 2 → she is a good girl
Sentence 3 → boy and girl are good

→ Remove stop words

Sentence 1 → good boy
Sentence 2 → good girl
Sentence 3 → boy girl good

	good	boy	girl	Output/label
Sentence 1	1	1	0	
Sentence 2	1	0	1	
Sentence 3	1	1	1	

What are the disadvantages of a bag of words?

1 - Frequency is the measure :

Frequency measures how many times each word appears in a sentence, with numbers like 0, 1, 2, 3 showing this. A word repeated often, like 4 times, is considered very important. However, a word that appears only once can still be important for the sentence's meaning, so it's given a 1. Other models, like TF-IDF, show how important each word is in a different way.

2 - There is no semantic understanding:

It happens that a lot of information is lost because it deals with words individually, meaning that all the words are independent, so the meaning is sometimes lost.

What are the disadvantages of a bag of words ?

3 - Sparse representation :

You'll find that your matrix in the end has a lot of zeros if your documentation is large.

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 2 \\ 9 & 0 & 0 & 6 \\ 7 & 0 & 0 & 0 \end{bmatrix}$$

What are the disadvantages of a bag of words ?

4 -The inability to deal with negation :

This means that it treats negative words as if they were normal words, which leads to losing a big part of the meaning.

5- N-GRAM

N-GRAM:

- Thinking of dealing with each word individually isn't the best because it makes me lose the sentence's meaning.

- For example, if we take each word separately, "book" could mean a book in both instances. However, when it's used with another word before or after it, the meaning changes. It could be understood as a verb, meaning to reserve, and the second instance would be understood as a book.

we need to book our tickets soon
we need to read this book soon

We have many types of N-GRAM :

This is Big Data AI Book

Uni-Gram	This	Is	Big	Data	AI	Book
Bi-Gram	This is	Is Big	Big Data	Data AI	AI Book	
Tri-Gram	This is Big	Is Big Data	Big Data AI	Data AI Book		

If there is a new word, consider it not existing at all, meaning don't assign any numbers to it at all, like the word "Hulk" in the second example.

```
v.transform(["Thor eat pizza"]).toarray()

array([[1, 1, 0, 0, 0, 1, 0, 1, 1]], dtype=int64)
```

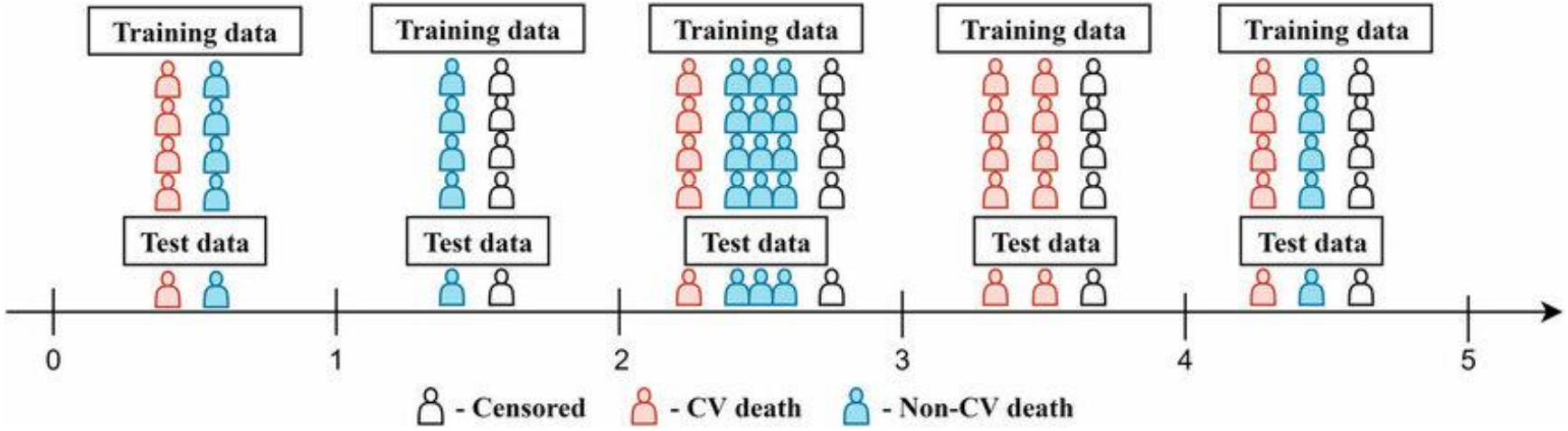
```
v.transform(["Hulk eat pizza"]).toarray()

array([[1, 1, 0, 0, 0, 1, 0, 0, 0]], dtype=int64)
```

	0 eat	1 eat pizza	2 loki	3 loki eat	4 loki tall	5 pizza	6 tall	7 thor	8 thor eat
Thor eat pizza	1	1	0	0	0	1	0	1	1
Hulk eat pizza	1	1	0	0	0	1	0	0	0

Coding note :

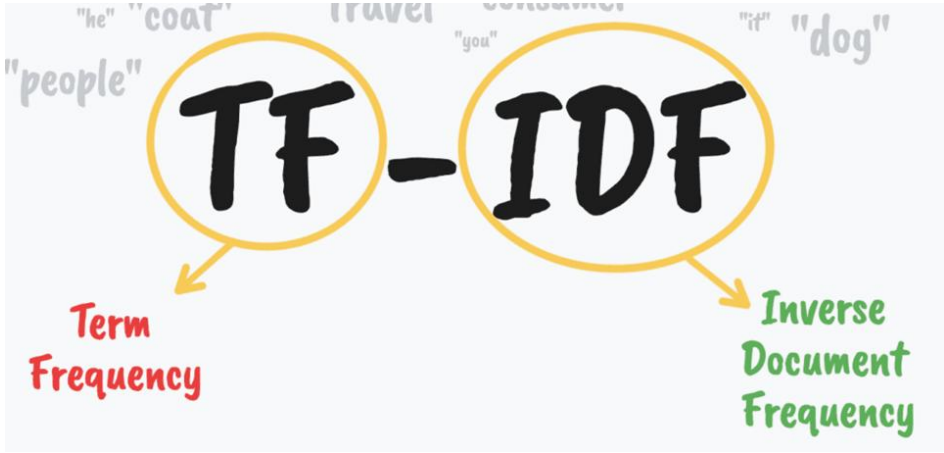
When splitting data for training and testing, "stratified" means ensuring that each group in the data is included in both sets. This is important because without it, the data could be split randomly, leading to all data from one group (e.g., group zero) being in the training set and all data from another group (e.g., group one) being in the testing set. Stratified splitting helps prevent this by balancing the groups in both sets.

.



```python
from sklearn.model_selection import train_test_split
import pandas as pd
X = data.drop('target_column', axis=1)  # Features
y = data['target_column']  # Target variable

# Split the dataset into train and test sets, stratifying by 'y'
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)


```

6 -TF-IDF


"he"
"coat"
travel
"you"
"it"
"dog"
"people"
TF-IDF
Term Frequency
Inverse Document Frequency

TF-IDF :

- TF-IDF stands for Term Frequency - Inverse Document Frequency

- We previously discussed using BOW and N-GRAM to represent my text as vectors.

- It has a big advantage over the BOW in that it focuses more on the weight in the sentence, meaning that the most influential word in the sentence will have a higher value or number than the other words.

- The "bag of words" (BOW) method worked on the frequency, meaning that words that are repeated often have a higher value, which is not correct because we find that rare words have more impact on the meaning of the sentence. Also, stopwords like "the," "and," and "or" are not influential at all. So, I used to remove them before applying the BOW method. TF-IDF gives a lower value to frequently repeated words because they are not as influential as stopwords

How TF-IDF works :

- It calculates the TF for each unique word, which is the number of times each word appears in the sentence divided by the total number of words in that sentence.
- To get the IDF for each unique word, we calculate the logarithm of the total number of sentences I have divided by the number of sentences containing that word.
- After that, multiply each column in the TF by the IDF column to get the final vectors.
- In the end, I will have a set of features that are the unique words in the form of vectors.

Example

- Sen 1 : good boy
- Sen 2 : good girl
- Sen 3 : boy gilr good
- First we calculate TF → TF = $\frac{\text{No.of rep in a sen}}{\text{No.of words in a sen}}$
- IDF = $\log\left(\frac{\text{no. of } sentances}{\text{No.of sentences containing words}}\right)$

	sen1	sen2	sen3
Good	1/2	1/2	1/3
Boy	1/2	0	1/3
girl	0	1/2	1/3

X

words	IDF
Good	Log(3/3)=0
Boy	Log(3/2)
girl	Log(3/2)

	good	boy	girl	output
Sen 1	0	½*log(3/2)	0	
Sen 2	0	0	½*log(3/2)	
Sen 3	0	1/3*log(3/2)	1/3*log(3/2)	

Here are some notes:

1- I found here the FEATURES that I will work on, which are the UNIQUE WORDS, and also I found them arranged alphabetically.

2- I notice that the most repeated value like "bird" for example, is given a small value, and a word like "brown" with less repetition is given a larger value.

```
   actions      bird     brown   catches       dog     early       fox  \
0 0.000000  0.000000  0.324676  0.000000  0.324676  0.000000  0.324676
1 0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
2 0.000000  0.392644  0.000000  0.392644  0.000000  0.392644  0.000000
3 0.447214  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000

        in     jumps      lazy  ...      over     quick     saves     speak  \
0 0.000000  0.324676  0.324676  ...  0.324676  0.324676  0.000000  0.000000
1 0.447214  0.000000  0.000000  ...  0.000000  0.000000  0.447214  0.000000
2 0.000000  0.000000  0.000000  ...  0.000000  0.000000  0.000000  0.000000
3 0.000000  0.000000  0.000000  ...  0.000000  0.000000  0.000000  0.447214

     stitch      than       the      time     words      worm
0 0.000000  0.000000  0.511956  0.000000  0.000000  0.000000
1 0.447214  0.000000  0.000000  0.447214  0.000000  0.000000
2 0.000000  0.000000  0.619130  0.000000  0.000000  0.392644
3 0.000000  0.447214  0.000000  0.000000  0.447214  0.000000

[4 rows x 22 columns]
```

7 - One hot encoding

id	color
1	red
2	blue
3	green
4	blue

One Hot Encoding

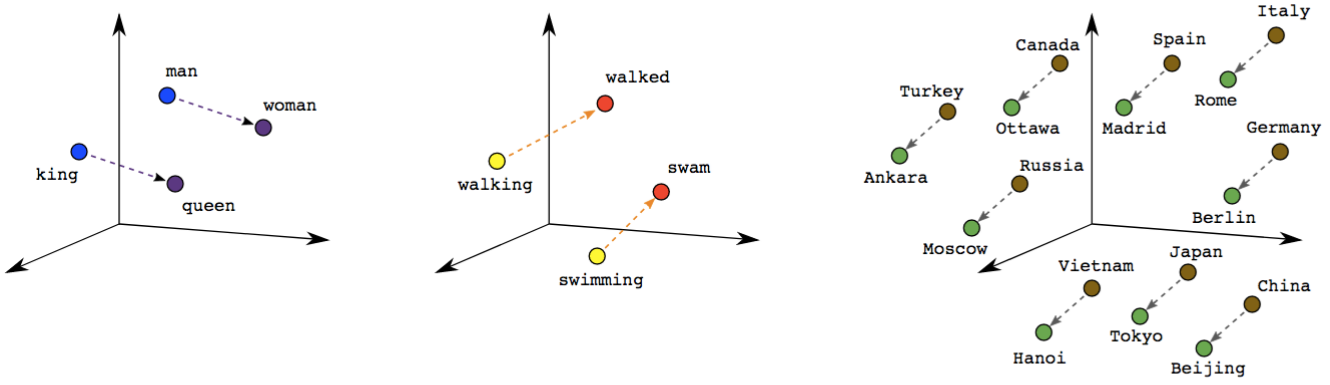
id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

One hot encoding isn't good for representing data with large sizes why ?

Let's assume that my vocabulary has 10k unique words. Using the one-hot encoding method, for example, when the word "man" is encountered, it will look at its index and set it to 1, while all other indices (from 0 to 9999) will be set to 0. This approach creates three problems:

1- "It will show me a very large number of features."
2- "It causes a problem with the sparse matrix, meaning that most of the values are zeros and a very few number are ones."
3- "It won't preserve the semantic for me. Why? Because if a word, for example, 'orange, apple,' are both very close to each other, but if one comes first and the other comes last, you will find that their vectors are completely different from each other."

8 - Word Embedding



Male-Female

Verb Tense

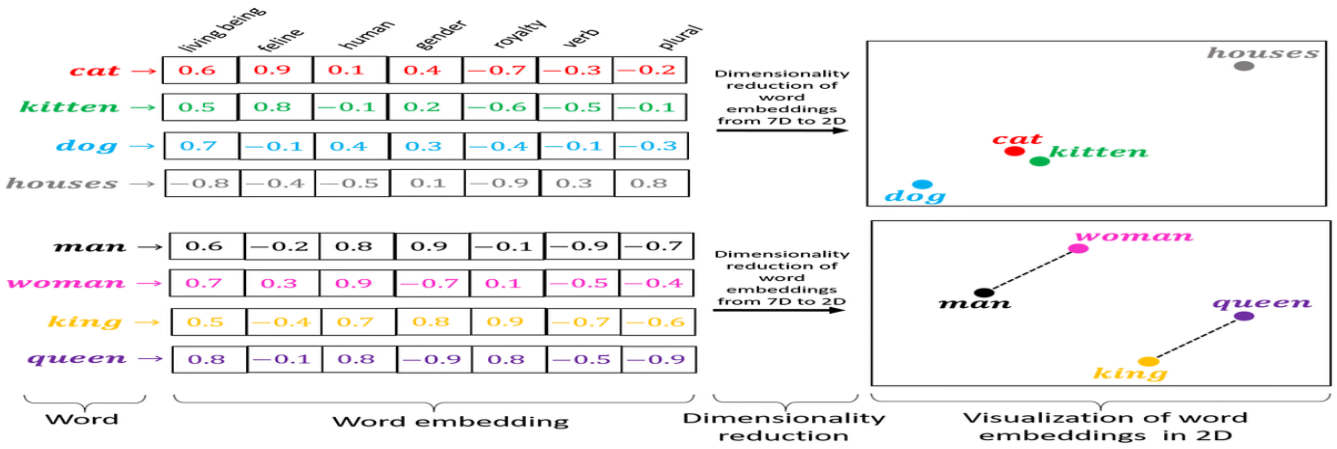
Country-Capital

Word Embedding :

- It's about algorithms I use to represent my words as vectors.

- But it's real advantage is that it can handle semantics, meaning words that have the same meaning or similar context. It gives them vectors that are close to each other, like apple, orange, and banana, which are all considered fruits. So you'll find that their vectors are close to each other, or like man and woman, and so on.

- I apply it using neural network models like Word2Vec, Glove, and FastText.



- let's see how word embedding works (solving the above 3 problems in One hot encoding):

- First, it creates a one-hot encoding for all the words I have, meaning each word is represented by a vector equal to the number of words in my corpus. All elements in the vector are zeros except for the index of the word, which is set to 1.

- After that, he takes the output of the one-hot encoding and feeds it as input to the embedding layer, which works as follows: It infers the characteristics between all the words and puts them as rows, such as GENDER, ROYAL, AGE

- The algorithm assigns a number to each word based on whether the word is related to that property or not. So, we find that a word like "BOY" is related to "Gender," so it gets a high number. But a word like "apple" is not related, so it gets a zero. Why then is "girl" equal -1 even though there is a strong relationship since it is the opposite of "boy”.

- Be careful when you determine the number If I say the dimension=10, it means I want to have 10 features (which are the attributes like Gender, Royal, ... that I already have

Man index 5k
Woman index 9k

man	woman
0	0
0	0
0	0
1	0
0	0
0	0
0	1

10k dim

Word embedding (feature representation)

	2000	5000	6000	9000	1000	7000
	Boy	Girl	king	queen	apple	mango
Gender	-1	1	-0.92	0.93	0.0	0.1
Royal	0.01	0.02	0.95	0.96	-0.02	0.01
Age	0.5	0.5	0.7	0.6	0.95	0.92
Food						
.						
.						

10 dim

Summary:

1- You define the vocab size and then get the indices of the words you have, which are the ones we will embedding to convert them into vectors, which are the features.

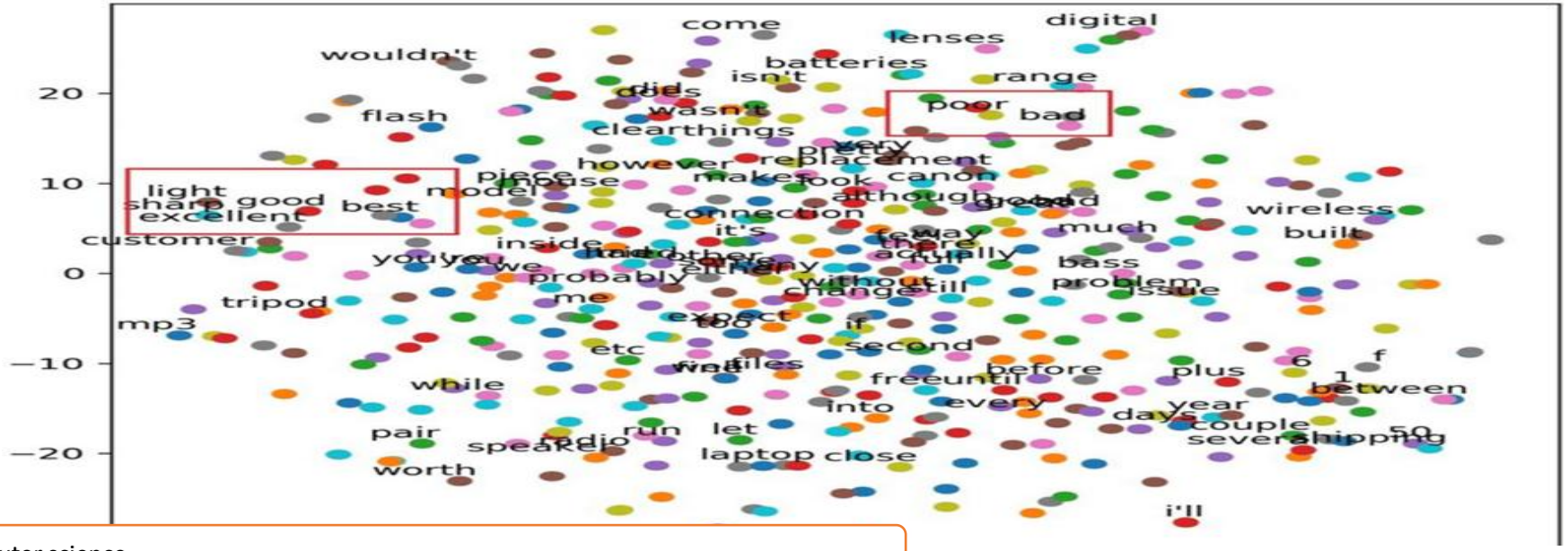
2- Also, we evaluate the word embedding through what???

Word similarity and word analogy

Notes:

1- The evidence that word embedding converts words into vectors while considering the semantic meaning, meaning that words with similar meanings will have close vectors.

2- In this image, the words that have undergone word embedding and then PCA are shown. If we look at words like best, good, excellent, we will find them very close to each other.



One-hot Encoded Words

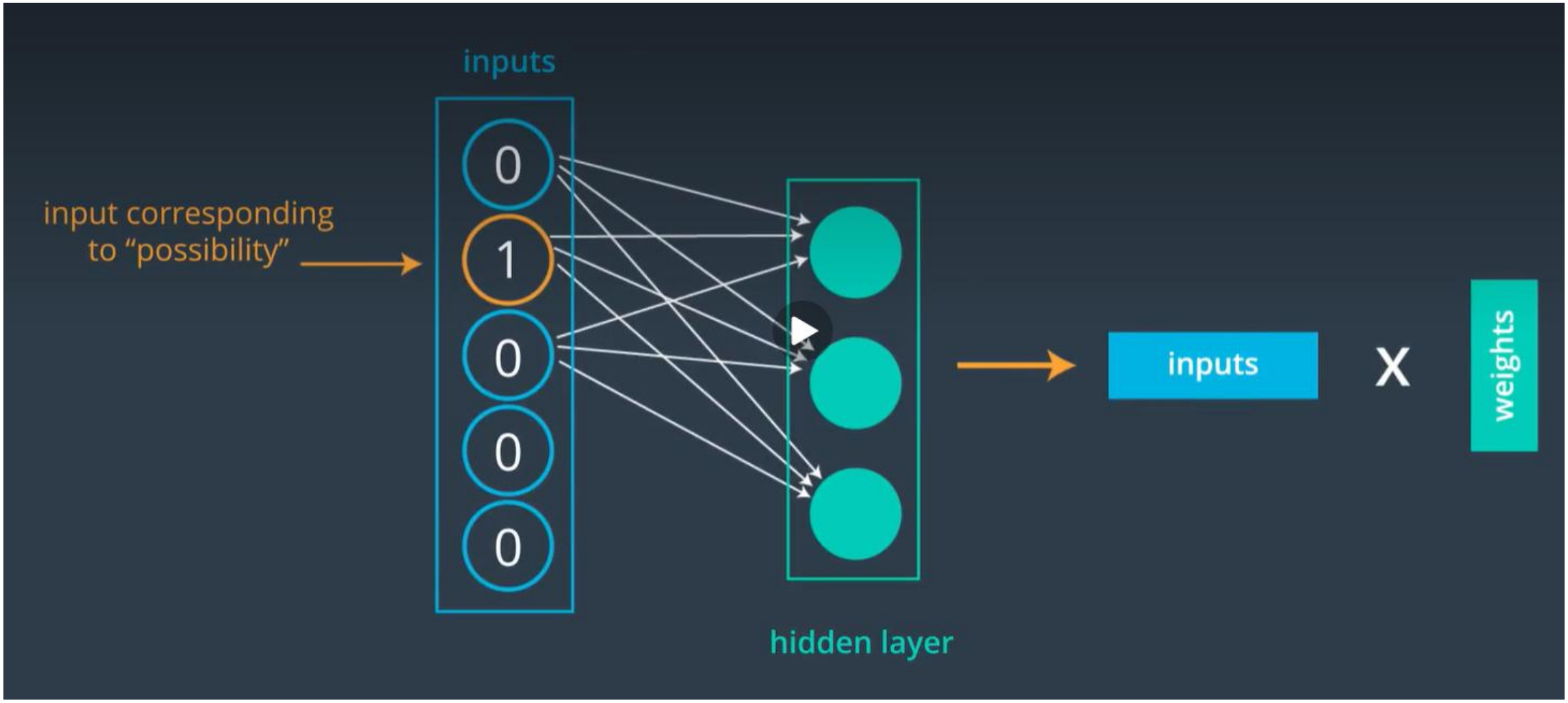
thousands of values long

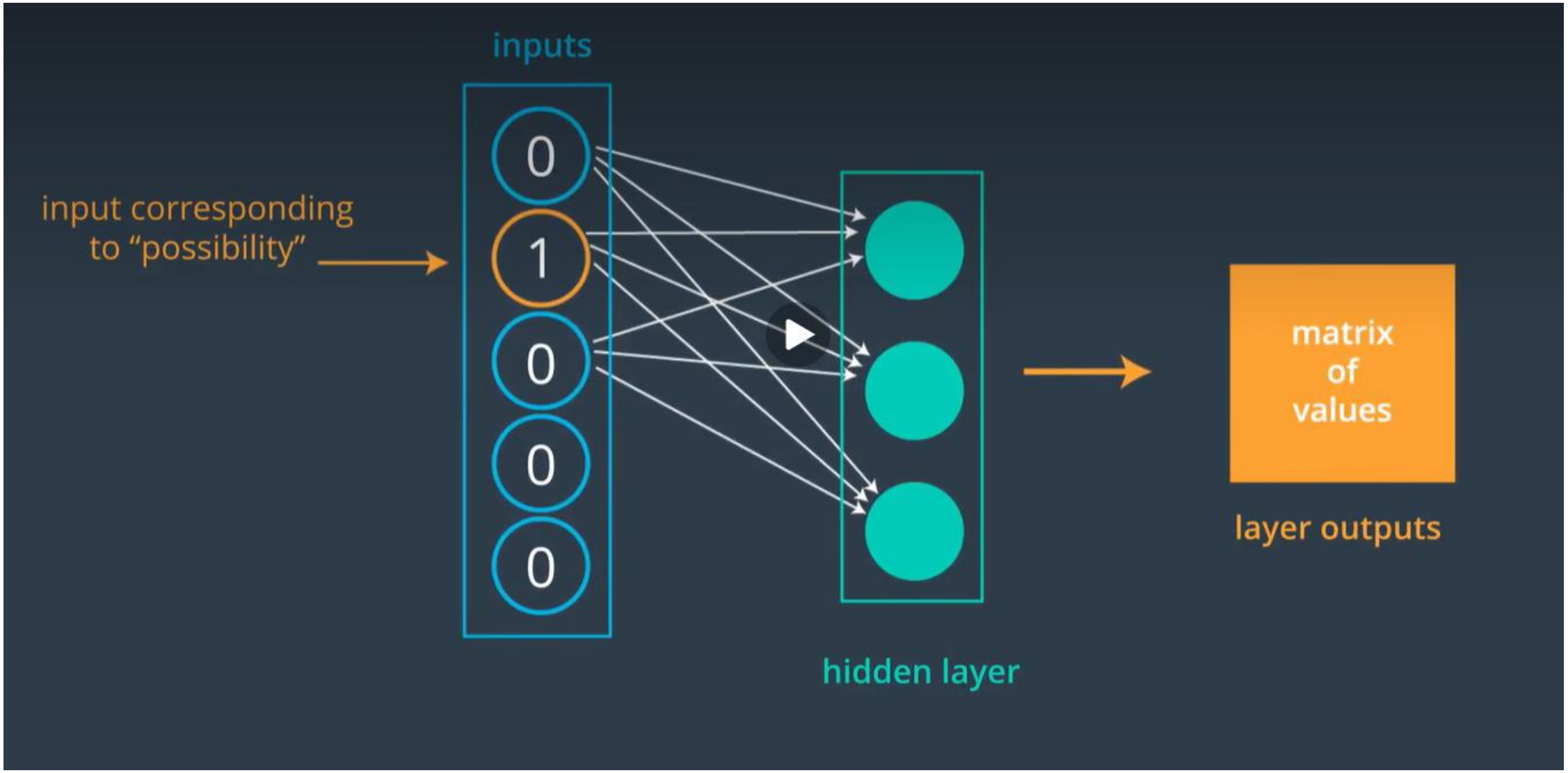
0
0
0
0
...
...
...
0
1
0
0
0
...
...
...
0

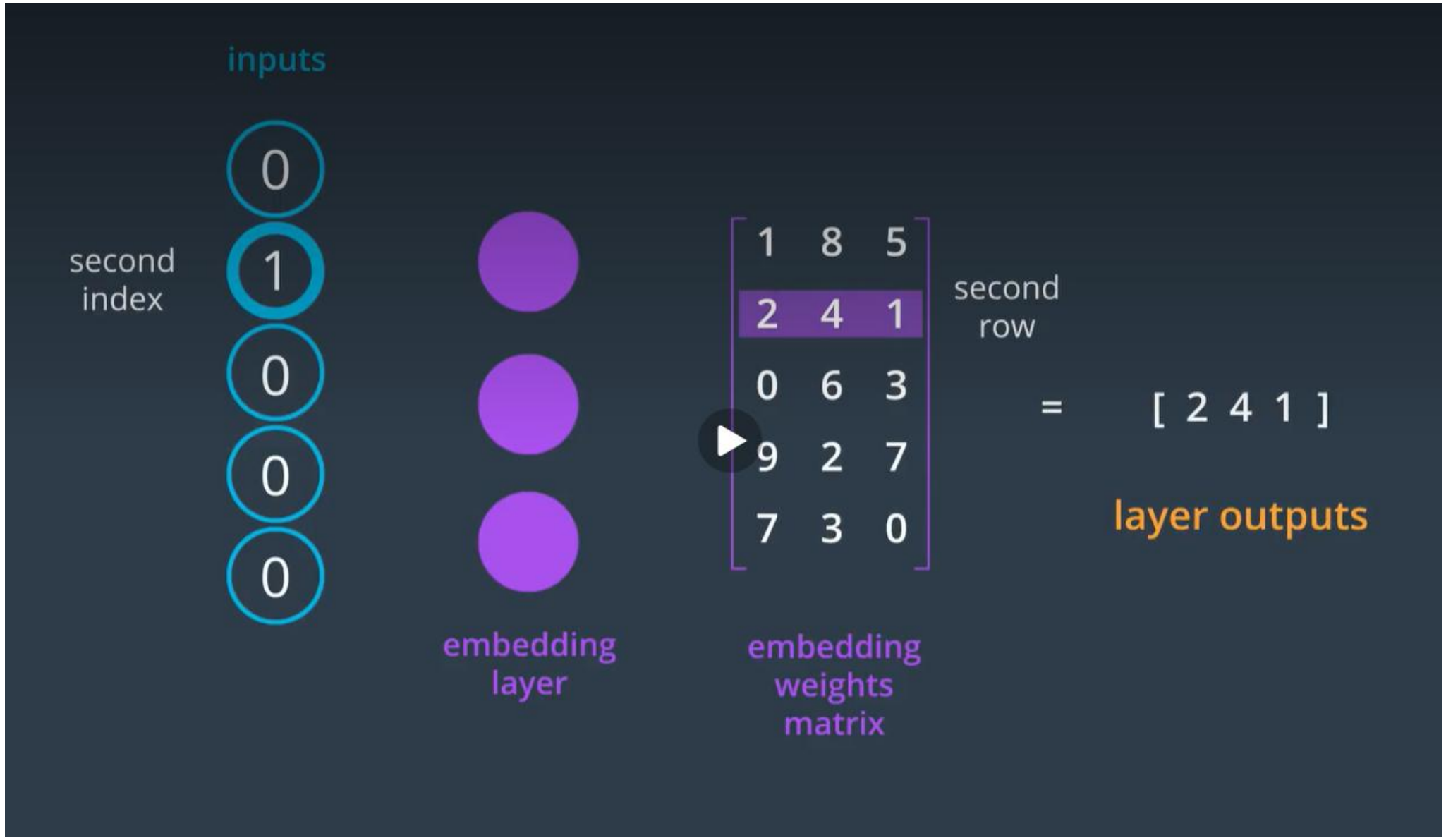
There was no possibility of taking a walk that day.

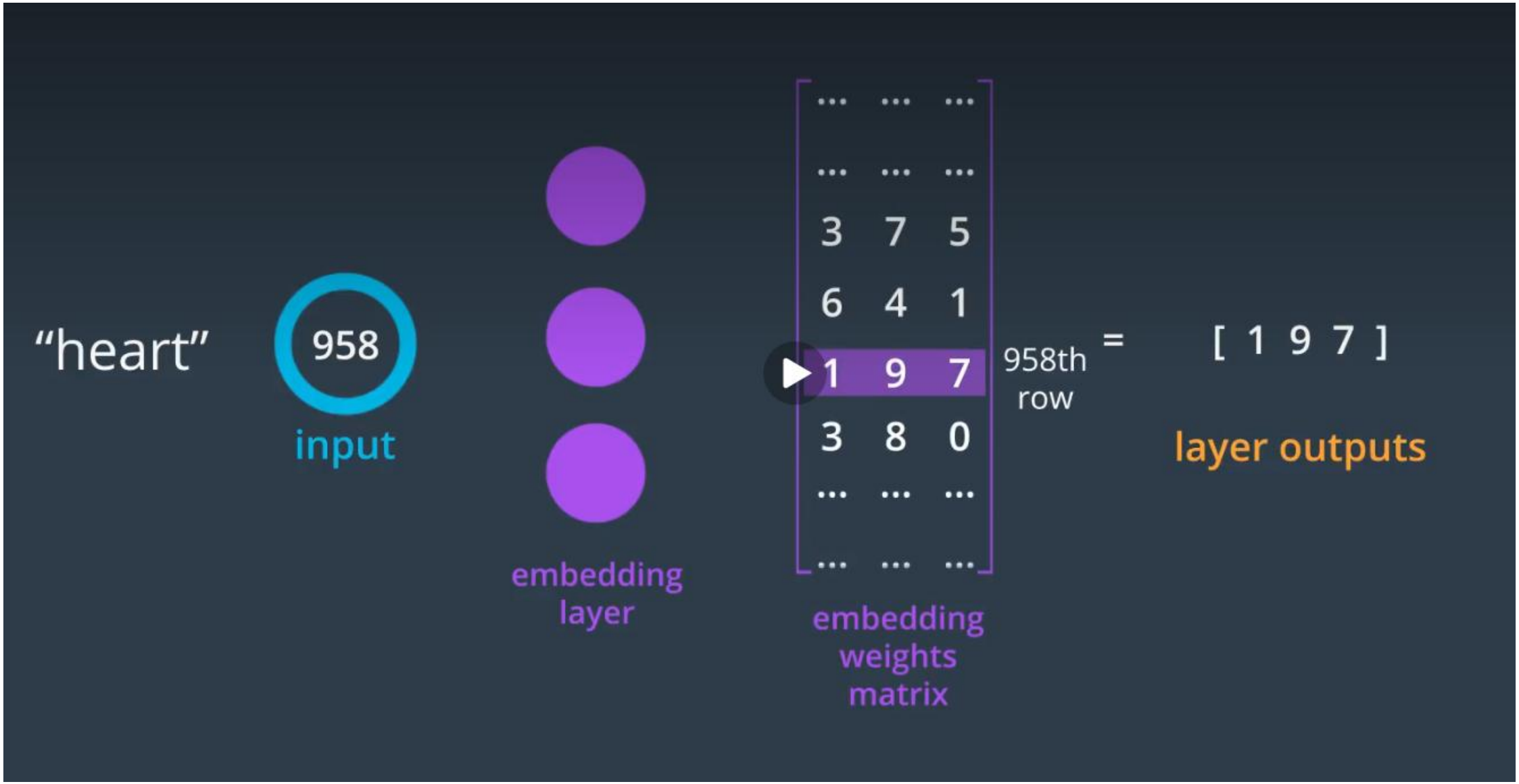
- first sentence of Jane Eyre by Charlotte Brontë

Large Vocabulary

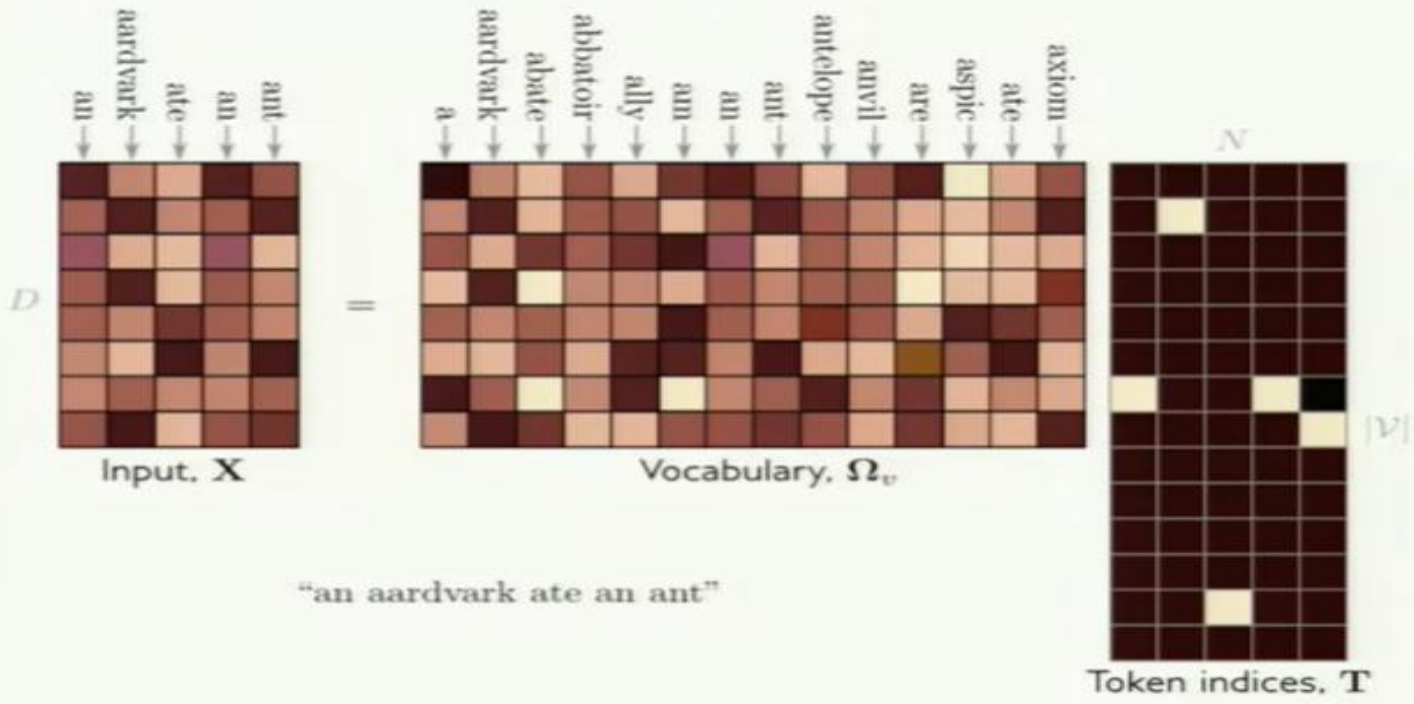

inputs
input corresponding
to "possibility"
0
1
0
0
0
hidden layer
inputs
X
weights


inputs
input corresponding
to "possibility"
0
1
0
0
0
hidden layer
matrix
of
values
layer outputs


inputs
second index
0
1
0
0
0
embedding layer
1 8 5
2 4 1
0 6 3
9 2 7
7 3 0
embedding weights matrix
second row
=
[2 4 1]
layer outputs


"heart"
958
input
embedding layer
... ...
... ...
3 7 5
6 4 1
1 9 7
3 8 0
... ...
... ...
embedding weights matrix
958th row
=
[1 9 7]
layer outputs

Word embedding



"an aardvark ate an ant"

D=1024, |V|=30522 for most tasks

Thank You